

Hyper-V Proxmox VM 불러오기 절차

이 문서는 Proxmox VM zip 파일을 아래 위치에 압축 해제한 상태를 기준으로 합니다.

```
바탕화면\a
```

현재 `a` 폴더 안에 아래 폴더들이 있는 상태입니다.

```
a
├─ Snapshots
├─ Virtual Hard Disks
└─ Virtual Machines
```

목표는 압축 해제한 원본 `.vhdx` / `.avhdx` 체크포인트 체인은 건드리지 않고, `a` 폴더 안에 새 작업 폴더를 만들어 `run.vhdx` 파일에만 변경분이 쌓이게 하는 것입니다.

주의사항

- 바탕화면\`a` 안의 `.vhdx`, `.avhdx` 파일을 개별 삭제하지 마세요.
- 원본 `.avhdx`를 VM에 직접 붙여서 부팅하지 마세요.
- 이 복구용 VM에서는 Hyper-V 검사점을 켜지 마세요.
- `run.vhdx`는 VM을 켜면 커질 수 있습니다. 다만 원본 체크포인트 체인 파일이 아니라 `a\proxmox-work\run.vhdx`만 커지게 하는 방식입니다.

1. 관리자 PowerShell 열기

시작 메뉴에서 PowerShell을 검색한 뒤 관리자 권한으로 실행합니다.

2. 경로 변수 설정

아래 명령어는 현재 Windows 사용자의 바탕화면 경로를 자동으로 잡습니다. 바탕화면이 OneDrive로 연결된 경우에도 보통 자동으로 맞습니다.

```
$Original = Join-Path ([Environment]::GetFolderPath("Desktop")) "a"
$Work = Join-Path $Original "proxmox-work"
$RunDisk = Join-Path $Work "run.vhdx"

$Original
$Work
$RunDisk
```

출력된 `$Original` 경로가 `Snapshots`, `Virtual Hard Disks`, `Virtual Machines`가 들어있는 `a` 폴더인지 확인합니다.

3. 최신 `.avhdx` 찾기

먼저 최신 `.avhdx` 목록을 확인합니다.

```
Get-ChildItem $Original -Recurse -File |
Where-Object { $_.Extension -eq ".avhdx" } |
Sort-Object LastWriteTime -Descending |
Select-Object -First 20 LastWriteTime, @{Name="GB";Expression=
{[math]::Round($_.Length / 1GB, 2)}}, FullName
```

가장 최신 `.avhdx` 를 부모 디스크로 저장합니다.

```
$Parent = Get-ChildItem $Original -Recurse -File |
Where-Object { $_.Extension -eq ".avhdx" } |
Sort-Object LastWriteTime -Descending |
Select-Object -First 1

$Parent.FullName
```

원본 `.avhdx` 와 같은 폴더에 `.vhdx` 하드링크를 만들 경로도 준비합니다. 하드링크를 다른 폴더에 만들면 상대 부모 경로가 깨져 `Test-VHD` 가 실패할 수 있으므로, 반드시 `$Parent.DirectoryName` 을 사용합니다.

```
$ParentLink = Join-Path $Parent.DirectoryName "parent-link.vhdx"
$ParentLink
```

출력이 비어 있으면 여기서 멈추고, 압축 해제한 VM에 체크포인트 파일이 실제로 있는지 다시 확인해야 합니다.

압축 해제로 날짜가 바뀐 경우 수동 지정

zip을 풀면서 모든 파일의 수정 시간이 압축 해제 시점으로 바뀌면, 위의 "최신 파일" 자동 선택이 틀릴 수 있습니다. 이 경우 파일 탐색기에서 실제로 사용할 `.avhdx` 경로를 확인한 뒤 직접 `$Parent`에 넣습니다.

예시:

```
$ParentPath = "C:\Users\사용자이름\Desktop\a\Snapshots\실제사용할파일.avhdx"
$Parent = Get-Item $ParentPath
$ParentLink = Join-Path $Parent.DirectoryName "parent-link.vhdx"
$Parent.FullName
$ParentLink
```

바탕화면 경로를 자동으로 조합하려면 아래처럼 쓸 수도 있습니다.

```
$ParentPath = Join-Path $Original "Snapshots\실제사용할파일.avhdx"
$Parent = Get-Item $ParentPath
$ParentLink = Join-Path $Parent.DirectoryName "parent-link.vhdx"
$Parent.FullName
$ParentLink
```

이후 단계에서는 `$Parent.FullName`을 직접 `New-VHD -ParentPath`에 넣지 않고, 4번에서 만드는 `$ParentLink`를 부모로 사용합니다.

4. 새 변경 디스크 만들기

작업용 폴더를 만듭니다.

```
New-Item -ItemType Directory -Path $Work -Force
```

`New-VHD -Differencing`은 부모 디스크 경로가 `.vhd` 또는 `.vhdx` 확장자여야 해서 `.avhdx`를 직접 부모로 넣으면 실패합니다.

따라서 원본 `.avhdx`와 같은 폴더에 해당 `.avhdx`를 가리키는 `.vhdx` 하드링크를 먼저 만듭니다. 하드링크는 같은 파일을 다른 이름으로 가리키는 것이므로, 실제 디스크 용량을 새로 거의 쓰지 않습니다.

```
Remove-Item $ParentLink -Force -ErrorAction SilentlyContinue
New-Item -ItemType HardLink -Path $ParentLink -Target $Parent.FullName
```

그 다음 이 `.vhdx` 하드링크를 부모로 하는 새 differencing disk를 만듭니다.

```
New-VHD -Path $RunDisk -ParentPath $ParentLink -Differencing
```

가상 하드 디스크의 체인이 끊어졌습니다 오류가 나는 경우

아래 오류가 나오면 선택한 `.avhdx` 가 자기 부모 디스크를 못 찾는 상태입니다.

```
가상 하드 디스크의 체인이 끊어졌습니다.
차이점 보관용 디스크의 부모 가상 하드 디스크를 찾을 수 없습니다. (0xC03A000D)
```

먼저 선택한 `.avhdx` 가 어떤 부모를 찾고 있는지 확인합니다.

```
$Info = Get-VHD -Path $Parent.FullName
$Info | Format-List Path, ParentPath, VhdType
Test-Path $Info.ParentPath
```

`Test-Path` 결과가 `False` 이면 부모 경로가 옛날 PC 경로이거나 현재 위치와 맞지 않는 것입니다.

부모 파일 이름을 뽑아서 `a` 폴더 안에서 같은 이름의 파일을 찾습니다.

```
$MissingParentName = Split-Path $Info.ParentPath -Leaf
Get-ChildItem $Original -Recurse -File |
Where-Object { $_.Name -eq $MissingParentName } |
Select-Object LastWriteTime, @{Name="GB";Expression=
{[math]::Round($_.Length / 1GB, 2)}}, FullName
```

나온 결과가 하나라면 그 경로를 실제 부모로 지정합니다.

```
$ActualParent = Get-ChildItem $Original -Recurse -File |
Where-Object { $_.Name -eq $MissingParentName } |
Select-Object -First 1

Set-VHD -Path $Parent.FullName -ParentPath $ActualParent.FullName
```

그 다음 다시 확인합니다.

```
Get-VHD -Path $Parent.FullName | Format-List Path, ParentPath, VhdType
```

만약 Set-VHD 에서 ID 불일치 오류가 나면, 정말 같은 체크포인트 체인의 부모 파일이 맞는지 확인한 뒤에만 아래 옵션을 사용합니다.

```
Set-VHD -Path $Parent.FullName -ParentPath $ActualParent.FullName -  
IgnoreIdMismatch
```

부모도 또 다른 부모를 못 찾을 수 있습니다. 그 경우 현재 부모 파일에 대해 같은 과정을 반복해서, 최종 .vhdx 까지 체인이 이어지게 해야 합니다.

체인 경로를 고친 뒤 다시 run.vhdx 를 만듭니다.

```
Remove-Item $RunDisk -Force -ErrorAction SilentlyContinue  
New-VHD -Path $RunDisk -ParentPath $ParentLink -Differencing
```

Test-Path 가 True 인데도 같은 오류가 나는 경우

Test-Path \$Info.ParentPath 가 True 여도, 바로 위 부모만 존재한다는 뜻입니다. 그 부모의 부모, 그 위의 부모 중 하나가 끊겨 있으면 New-VHD 에서 같은 0xC03A000D 오류가 날 수 있습니다.

먼저 Hyper-V가 전체 체인을 정상으로 보는지 확인합니다.

```
Test-VHD -Path $Parent.FullName  
Test-VHD -Path $ParentLink
```

하나라도 False 이면 전체 체인 중 어딘가가 끊긴 것입니다.

아래 명령으로 부모 체인을 끝까지 따라가면서 깨진 지점을 찾습니다.

```

$Current = $Parent.FullName

while ($Current) {
    Write-Host "`nCHECK:" $Current
    $Vhd = Get-VHD -Path $Current
    $Vhd | Format-List Path, ParentPath, VhdType

    if ([string]::IsNullOrEmpty($Vhd.ParentPath)) {
        Write-Host "END: base disk reached"
        break
    }

    if (-not (Test-Path $Vhd.ParentPath)) {
        Write-Host "BROKEN PARENT:" $Vhd.ParentPath
        $MissingName = Split-Path $Vhd.ParentPath -Leaf
        Get-ChildItem $Original -Recurse -File |
        Where-Object { $_.Name -eq $MissingName } |
        Select-Object LastWriteTime, @{Name="GB";Expression=[math]::Round($_.Length / 1GB, 2)}, FullName
        break
    }

    $Current = $Vhd.ParentPath
}

```

BROKEN PARENT 가 나오면, 출력된 파일 이름과 같은 파일을 a 폴더 안에서 찾아 현재 \$Current 디스크의 부모로 다시 지정합니다.

```

$ActualParent = Get-ChildItem $Original -Recurse -File |
Where-Object { $_.Name -eq $MissingName } |
Select-Object -First 1

Set-VHD -Path $Current -ParentPath $ActualParent.FullName

```

다시 전체 체인을 검사합니다.

```
Test-VHD -Path $Parent.FullName
```

True 가 나올 때까지 같은 과정을 반복합니다. 그 다음 run.vhdx 를 다시 만듭니다.

```
Remove-Item $RunDisk -Force -ErrorAction SilentlyContinue
New-VHD -Path $RunDisk -ParentPath $ParentLink -Differencing
```

만약 `Test-VHD -Path $Parent.FullName` 은 `True` 인데 `Test-VHD -Path $ParentLink` 만 `False` 이면, 이 PC의 Hyper-V가 `.avhdx` 하드링크 우회를 받아들이지 않는 상황일 수 있습니다. 이 경우 안전한 대안은 공간을 확보한 뒤 `Convert-VHD` 로 체인을 단일 `.vhd` 로 병합하거나, 원본 제공자에게 Hyper-V 내 보내기본을 다시 받는 것입니다.

이후 구조는 아래처럼 전부 `a` 폴더 안에서 처리됩니다.

```
바탕화면\a\Snapshots          = 원본 체크포인트 보존
바탕화면\a\Virtual Hard Disks = 원본 디스크 보존
바탕화면\a\Virtual Machines   = 원본 구성 파일 보존
선택한 avhdx와 같은 폴더\parent-link.vhdx = 부모 체크포인트를 가리키는 하드링크
바탕화면\a\proxmox-work\run.vhdx = 새 변경분 저장
```

문제가 생기면 VM을 끄고 `run.vhdx` 만 지운 뒤 다시 만들면 됩니다.

하드링크 생성이 실패하면 `$ParentLink` 가 `$Original` 아래에 있는지 확인합니다. 이 문서처럼 `$ParentLink` 를 선택한 `.avhdx` 와 같은 폴더에 만들면 보통 같은 드라이브라서 하드링크가 됩니다.

5. 이전 실패 VM 등록 제거

Hyper-V에 실패한 `Proxmox-VE` VM이 이미 있으면, 먼저 VM 등록만 제거합니다. 이 단계는 파일 삭제가 아니라 **Hyper-V 목록에서 실패한 VM만 지우는 작업**입니다.

```
Stop-VM -Name "Proxmox-VE" -TurnOff -Force -ErrorAction SilentlyContinue
Remove-VM SavedState -VMName "Proxmox-VE" -ErrorAction SilentlyContinue
Remove-VM -Name "Proxmox-VE" -Force -ErrorAction SilentlyContinue
```

중간에 "없음", "이미 꺼져 있음", "저장된 상태가 없음" 같은 경고가 나오면 무시해도 됩니다.

이 명령은 압축 해제한 `바탕화면\a` 폴더를 삭제하지 않습니다.

Hyper-V 관리자에서 직접 할 수도 있습니다.

1. Hyper-V 관리자 열기
2. VM 목록에 `Proxmox-VE` 가 있으면 우클릭

3. 삭제

4. 바탕화면\ a 폴더는 지우지 않기

6. run.vhdx 로 새 Hyper-V VM 만들기

이제 새 VM을 만듭니다. 이때 원본 .avhdx 나 .vhdx 를 붙이는 것이 아니라, 반드시 아래 파일을 붙입니다.

```
바탕화면\ a\proxmox-work\run.vhdx
```

처음 부팅/복구 확인용으로는 **8GB**부터 시작해도 됩니다.

```
New-VM `
-Name "Proxmox-VE" `
-Generation 2 `
-MemoryStartupBytes 8GB `
-VHDPath $RunDisk
```

메모리 기준:

- 32GB RAM PC
- 처음 부팅/복구 확인: 8GB 추천
- Proxmox 안에서 VM/LXC를 여러 개 돌릴 예정: 이후 12GB ~ 16GB 로 조정
- 호스트 Windows 안정성을 위해 처음부터 24GB 이상 할당은 비추천
- 128GB RAM PC
- 처음 부팅/복구 확인: 16GB ~ 32GB
- Proxmox 안 VM/LXC 여러 개 실행: 64GB 추천
- 많이 켜야 하면 80GB ~ 96GB 까지 가능
- Windows/Hyper-V용으로 최소 24GB ~ 32GB 정도는 남겨두는 것을 추천

나중에 메모리를 16GB로 바꾸려면 VM을 끈 뒤 실행합니다.

```
Set-VMemory -VMName "Proxmox-VE" -StartupBytes 16GB
```

128GB RAM PC에서 Proxmox에 64GB를 주려면:

```
Set-VMemory -VMName "Proxmox-VE" -StartupBytes 64GB
```


더 많이 필요하면 96GB까지 올릴 수 있습니다.

```
Set-VMemory -VMName "Proxmox-VE" -StartupBytes 96GB
```

7. 첫 부팅 전 검사점과 보안 부팅 끄기

반드시 VM을 시작하기 전에 아래를 실행합니다.

```
Set-VM -Name "Proxmox-VE" -AutomaticCheckpointsEnabled $false
Set-VM -Name "Proxmox-VE" -CheckpointType Disabled
Set-VMFirmware -VMName "Proxmox-VE" -EnableSecureBoot Off
Set-VMProcessor -VMName "Proxmox-VE" -CompatibilityForMigrationEnabled
$true
```

Hyper-V 관리자에서도 확인합니다.

- 설정 -> 검사점: 검사점 사용 안 함
- 설정 -> 보안: 보안 부팅 사용 안 함
- 설정 -> 펌웨어: 하드 디스크가 네트워크 부팅보다 위에 있음

8. 연결된 디스크가 run.vhdx 인지 확인

시작하기 전에 반드시 VM이 실제로 run.vhdx 를 물고 있는지 확인합니다.

```
Get-VMHardDiskDrive -VMName "Proxmox-VE" | Select-Object Path
```

정상 결과는 아래처럼 끝나야 합니다.

```
...\Desktop\a\proxmox-work\run.vhdx
```

아래처럼 원본 파일을 직접 물고 있으면 잘못된 상태입니다. 이 경우 VM을 시작하지 마세요.

```
...\Desktop\a\Snapshots\어떤파일.avhdx
...\Desktop\a\Virtual Hard Disks\어떤파일.vhdx
```

잘못 연결되어 있으면 VM을 삭제한 뒤 6번부터 다시 진행합니다.

9. 네트워크 없이 먼저 부팅

먼저 네트워크를 붙이지 않은 상태로 VM을 시작합니다.

```
Start-VM -Name "Proxmox-VE"
```

부팅되면 원본 체크포인트/디스크 파일이 빠르게 커지지 않는지 확인합니다. 정상적으로는 주로 아래 파일이 커집니다.

```
바탕화면\a\proxmox-work\run.vhdx
```

10. 부팅 안정 후 네트워크 연결

NAT 방식이면 Default Switch를 붙입니다.

```
Connect-VMNetworkAdapter -VMName "Proxmox-VE" -SwitchName "Default Switch"
```

공유기/물리 LAN에 직접 붙는 브리지 방식이 필요하다면 Hyper-V 관리자에서 External Switch를 만들고 연결합니다.

```
Connect-VMNetworkAdapter -VMName "Proxmox-VE" -SwitchName "External Switch"
```

External Switch 부분은 실제 만든 스위치 이름으로 바꿉니다.

11. Proxmox 네트워크 설정

Proxmox에서는 보통 실제 랜카드 역할의 `eth0`에는 IP를 붙이지 않고, 브리지인 `vbr0`에 IP를 붙입니다.

기본 원칙:

```
eth0  = Hyper-V 가상 랜카드, 보통 manual  
vbr0  = Proxmox 관리 IP와 내부 VM/LXC 브리지
```

따라서 Proxmox 웹 UI 접속 주소는 `eth0`가 아니라 `vbr0` IP를 사용합니다.

```
https://vbr0-IP:8006
```

Hyper-V 스위치별 추천

목적	Hyper-V 스위치	eth0	vbr0
Windows 호스트에서만 Proxmox 웹 접속	Default Switch	manual	dhcp
Proxmox 인터넷 임시 연결	Default Switch	manual	dhcp
같은 공유기망의 다른 PC/폰에서도 접속	External Switch	manual	static 추천
Proxmox 안 VM/LXC를 실제 LAN에 붙임	External Switch	manual	static 추천

vbr0 를 DHCP로 바꾸기

Default Switch를 쓸 때는 vbr0 를 DHCP로 두는 것이 편합니다.

Proxmox 콘솔에서 파일을 엽니다.

```
nano /etc/network/interfaces
```

아래처럼 설정합니다.

```
auto lo
iface lo inet loopback

auto eth0
iface eth0 inet manual

auto vbr0
iface vbr0 inet dhcp
    bridge-ports eth0
    bridge-stp off
    bridge-fd 0
```

저장 후 적용합니다.

```
ifreload -a
```

ifreload 가 없거나 실패하면 재부팅합니다.

```
reboot
```

부팅 후 IP를 확인합니다.

```
ip addr show vmbr0
```

inet 뒤에 나온 IP로 접속합니다.

```
https://해당-IP:8006
```

External Switch에서 static 예시

공유기 대역이 192.168.0.x, 게이트웨이가 192.168.0.1 이면 예시는 아래와 같습니다.

```
auto lo
iface lo inet loopback

auto eth0
iface eth0 inet manual

auto vmbr0
iface vmbr0 inet static
    address 192.168.0.50/24
    gateway 192.168.0.1
    bridge-ports eth0
    bridge-stp off
    bridge-fd 0
```

접속:

```
https://192.168.0.50:8006
```

12. Proxmox 내부 VM에서 KVM 오류가 나는 경우

Proxmox 안의 VM을 시작할 때 아래 오류가 나오면, Hyper-V VM에 nested virtualization 이 켜져 있지 않은 상태일 가능성이 큽니다.

TASK ERROR: KVM virtualisation configured, but not available.
Either disable in VM configuration or enable in BIOS.

Hyper-V 안에서 Proxmox를 돌리고, 그 Proxmox 안에서 다시 VM을 돌리는 구조이므로 Windows Hyper-V VM에 CPU 가상화 확장을 노출해야 합니다.

Windows 관리자 PowerShell에서 `Proxmox-VE` VM을 끈 뒤 실행합니다.

```
Stop-VM -Name "Proxmox-VE" -TurnOff -Force
Set-VMProcessor -VMName "Proxmox-VE" -ExposeVirtualizationExtensions $true
Set-VMemory -VMName "Proxmox-VE" -DynamicMemoryEnabled $false
Set-VMNetworkAdapter -VMName "Proxmox-VE" -MacAddressSpoofing On
Start-VM -Name "Proxmox-VE"
```

각 설정의 의미:

- `ExposeVirtualizationExtensions`: Proxmox 안에서 KVM/VT-x/AMD-V가 보이게 함
- `DynamicMemoryEnabled $false`: nested virtualization 안정성을 위해 동적 메모리 끄
- `MacAddressSpoofing On`: Proxmox 안 VM/LXC가 브리지 네트워크를 쓸 수 있게 함

Proxmox 콘솔에서 KVM이 보이는지 확인합니다.

```
egrep -c '(vmx|svm)' /proc/cpuinfo
ls -l /dev/kvm
```

`egrep` 결과가 1 이상이고 `/dev/kvm`이 보이면 KVM을 사용할 수 있습니다.

Host doesn't support requested feature ... aes 오류

Proxmox 내부 VM을 시작할 때 아래처럼 나오면 KVM 자체가 없는 문제가 아니라, 해당 VM의 CPU 타입이 현재 Hyper-V 중첩 환경에서 보이지 않는 CPU 기능을 요구하는 문제입니다.

```
kvm: warning: host doesn't support requested feature: CPUID[...].ECX.aes
kvm: Host doesn't support requested features
start failed: QEMU exited with code 1
```

예를 들어 VM 설정에 아래처럼 되어 있을 수 있습니다.

```
cpu: x86-64-v2-AES
```

이 경우 AES 요구가 없는 CPU 타입으로 낮춘 뒤 다시 시작합니다. 예를 들어 VM ID가 120 이면:

```
qm set 120 --cpu x86-64-v2
qm start 120
```

VM ID가 199 에서 같은 오류가 나도 동일하게 처리합니다.

```
qm set 199 --cpu x86-64-v2
qm start 199
```

그래도 실패하면 더 보수적인 CPU 타입으로 바꿉니다.

```
qm set 120 --cpu kvm64
qm start 120
```

GUI에서는:

```
VM 120 -> Hardware -> Processors -> Type
```

에서 x86-64-v2-AES 대신 x86-64-v2 또는 kvm64 로 변경합니다.

같은 문제가 있는 VM을 찾으려면:

```
grep -R "^cpu: .*AES" /etc/pve/qemu-server/*.conf
```

x86-64-v2-AES 가 들어간 VM들을 한 번에 x86-64-v2 로 바꾸려면:

```
for conf in /etc/pve/qemu-server/*.conf; do
  id=$(basename "$conf" .conf)
  if grep -q "^cpu: .*AES" "$conf"; then
    echo "Updating VM $id"
    qm set "$id" --cpu x86-64-v2
  fi
done
```

이후 필요한 VM을 다시 시작합니다.

```
qm start 199
```

Hyper-V 가상 프로세서 오류 또는 Proxmox 재부팅이 반복되는 경우

Windows Hyper-V 이벤트 로그에 아래와 비슷한 18550 위험 이벤트가 나오고 Proxmox가 재부팅되면, Hyper-V nested virtualization 쪽의 가상 CPU 안정성 문제일 수 있습니다.

가상 프로세서 레지스터에 액세스하는 동안 복구할 수 없는 오류가 발생...

이 경우 Proxmox-VE VM을 끈 뒤 Windows 관리자 PowerShell에서 vCPU 수를 줄이고 CPU 호환성 옵션을 끕니다.

```
Stop-VM -Name "Proxmox-VE" -TurnOff -Force

Set-VMProcessor -VMName "Proxmox-VE" -Count 16
Set-VMProcessor -VMName "Proxmox-VE" -ExposeVirtualizationExtensions $true
Set-VMProcessor -VMName "Proxmox-VE" -CompatibilityForMigrationEnabled
>false

Set-VMemory -VMName "Proxmox-VE" -DynamicMemoryEnabled $false
Set-VMemory -VMName "Proxmox-VE" -StartupBytes 64GB

Start-VM -Name "Proxmox-VE"
```

설정이 적용되었는지 Windows에서 확인합니다.

```
Get-VMProcessor -VMName "Proxmox-VE" | Format-List
Count,ExposeVirtualizationExtensions,CompatibilityForMigrationEnabled
Get-VMemory -VMName "Proxmox-VE" | Format-List
DynamicMemoryEnabled,Startup
```

예상:

```
Count                : 16
ExposeVirtualizationExtensions : True
CompatibilityForMigrationEnabled : False
DynamicMemoryEnabled  : False
Startup               : 68719476736
```

Proxmox 안에서는 vCPU 개수를 `egrep -c '(vmx|svm)' /proc/cpuinfo` 로 판단하지 않습니다. 이 명령은 CPU 개수가 아니라 `vmx / svm` 문자열이 몇 번 나오는지 세는 것이므로, vCPU가 16개여도 32처럼 보일 수 있습니다.

vCPU 개수는 아래 명령으로 확인합니다.

```
nproc  
lscpu | grep -E '^CPU\(s\):|Thread|Core|Socket'
```

KVM 사용 가능 여부는 아래로 확인합니다.

```
ls -l /dev/kvm
```

내부 VM은 하나씩 천천히 시작

Proxmox 안의 VM을 여러 개 한 번에 켜면 Hyper-V nested virtualization 환경에서 상태 조화가 꼬이거나 Proxmox VM이 재부팅될 수 있습니다. 하나씩 시작하고 잠깐 기다린 뒤 다음 VM을 켵니다.

예:

```
qm start 100  
sleep 20  
qm status 100  
  
qm start 110  
sleep 20  
qm status 110  
  
qm start 120  
sleep 20  
qm status 120  
  
qm start 130  
sleep 20  
qm status 130
```

이 방식으로 정상 동작하면, 문제는 특정 VM 자체보다 여러 VM을 동시에 시작하면서 생기는 nested virtualization/상태 갱신 불안정일 가능성이 큼니다.

그래도 안 되면 실제 PC BIOS/UEFI에서 아래 설정이 켜져 있는지 확인합니다.

- Intel: Intel VT-x
- AMD: SVM Mode 또는 AMD-V
- 가능하면 VT-d/IOMMU

특정 VM만 임시로 켜야 하는 경우

KVM을 끄면 느리지만 해당 VM이 켜질 수 있습니다. 예를 들어 VM ID가 110 이면 Proxmox 콘솔에서:

```
qm set 110 --kvm 0
qm start 110
```

GUI에서는:

```
VM 110 -> Options -> KVM hardware virtualization -> No
```

VM 110 not running (500) 메시지

아래 메시지는 VM 110이 실제로 실행 중이 아닌 상태에서 콘솔 열기, 정지, 재시작 같은 작업을 했을 때 나올 수 있습니다.

```
VM 110 not running (500)
```

KVM 오류 때문에 시작에 실패한 뒤 이 메시지가 이어서 나오는 경우가 많습니다. 먼저 nested virtualization을 켜고 VM 110을 다시 시작합니다.

```
qm status 110
qm start 110
```

13. 이번 시도 변경분만 초기화하기

이번 부팅 시도를 버리고 처음 상태로 되돌리고 싶으면, VM을 끄고 `run.vhdx` 만 다시 만듭니다.

```
Stop-VM -Name "Proxmox-VE" -TurnOff -Force
Remove-VM -Name "Proxmox-VE" -Force
Remove-Item $RunDisk -Force
New-VHD -Path $RunDisk -ParentPath $ParentLink -Differencing
```

그 다음 6번부터 다시 진행합니다.

예상 동작

- VM을 켜면 `run.vhdx` 는 커질 수 있습니다.
- 원본 바탕화면\a\Snapshots, 바탕화면\a\Virtual Hard Disks, 바탕화면\a\Virtual Machines 는 체크포인트 원본 체인으로 보존합니다.
- 새 변경분은 바탕화면\a\proxmox-work\run.vhdx 에 저장합니다.
- C드라이브 여유 공간이 부족하면 부팅 전에 먼저 공간을 확보해야 합니다.

14. 사용한 PowerShell 변수 정리

작업이 끝난 뒤 현재 PowerShell 창에서 사용한 변수를 지우고 싶으면 아래를 실행합니다.

```
Remove-Variable Original, Work, RunDisk, Parent, ParentPath, ParentLink -
ErrorAction SilentlyContinue
```

이 명령은 PowerShell 변수만 지우며, VM 파일이나 디스크 파일은 삭제하지 않습니다.